

Publication de documents HTML

Pascal CABAUD
Pascal.Cabaud@wanadoo.fr

27 avril 2001

Résumé

Ces quelques pages forment le support d'un cours donné à l'Université Paris VII à des étudiants en DESS spécialistes de traduction.

Ces derniers n'étant pas des programmeurs aguerris, on dit quelques mots sur JavaScript sous forme de recettes de cuisine commentées.

Table des matières

1	Licence	3
2	Rappels divers	4
2.1	Réseaux internet et intranet	4
2.2	Protocoles de communication	4
2.3	Le modèle client-serveur	4
2.4	Langages à balises et formatage de textes	4
2.5	<i>World Wide Web</i>	6
2.6	Langages de programmation	6
2.7	<i>Common Gateway Interface</i> versus JavaScript	7
2.8	Synthèse	7
2.9	<PUB> À propos de ce poly... </PUB>	7
3	HTML élémentaire	8
3.1	La page minimale	8
3.2	Détail sur les balises	8
3.3	Structuration élémentaire	9
3.4	Un «petit» mot sur les liens	11
3.5	Un mot sur l'encodage	12
3.6	Styles élémentaires	12
3.7	Le premier mot de la fin...	12
4	Mise en page améliorée	13
4.1	Inclusion d'images	13
4.2	Listes	14
4.3	Tableaux	16
4.4	Polices, couleurs, tailles, styles	17
4.5	Derniers détails de mise en page	18

<i>TABLE DES MATIÈRES</i>	2
5 <i>Frames</i>	19
5.1 Déclaration	19
5.2 Les attributs de <i><FRAMESET></i>	19
6 <i>Cascading Style Sheets</i>	21
6.1 Usage, types	21
6.2 Syntaxe, classes de styles et héritage	21
7 Compléments divers	24
7.1 Ouvrir et fermer une nouvelle fenêtre...	24
7.2 Envoyer un <i>mail</i>	25
8 Bibliographie	26

1 Licence

Commeçons par les choses qui fâchent :

Copyright (c) 2000-2001, Pascal CABAUD, Universite Paris 7
All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the Universite Paris 7 nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Apple, AT&T, Bell Labs, IBM, Java, JavaScript, Macintosh, Microsoft¹, Netscape, Sun Micro Systems, Windows², Unix sont des noms ou marques déposés par leurs propriétaires respectifs.

¹«Petit mou» pour les intimes.

²*Quine-daube* pour les connaisseurs.

2 Rappels divers

2.1 Réseaux internet et intranet

L'idée de faire communiquer des ordinateurs entre eux ne date pas d'hier. En effet, l'*US Air Force* avait envisagé la chose dès les années 60 (projet *DARPA*³) et sa mise en place date de 1975 (*ARPANET*). Quant à l'idée de réseaux, elle date de bien avant encore ! Citons [Kir99] :

« La notion de réseau est probablement aussi ancienne que le besoin de communiquer à distance. Transportons nous à l'âge de pierre et imaginons que les hommes s'envoyaient des messages en frappant sur des tambours [...]. Supposons que l'homme des cavernes A veuille inviter son copain B pour jouer à se lancer des cailloux, mais qu'ils habitent trop loin pour pouvoir entendre leur signaux sonnores respectifs. Quelles sont les solutions possibles ? Il pourrait 1) rendre visite à B, 2) prendre un tambour plus gros, 3) demander à C, qui habite à mi-chemin, de faire passer le message. Cette dernière solution s'appelle un réseau. »

Les choses ont bien changé depuis 1975 : les ordinateurs sont beaucoup plus puissants, les communications sont beaucoup moins chères et bien plus rapides, des normes de communication se sont imposées. Les besoins aussi ont changé. Aujourd'hui, on partage ses données avec son voisin de réseau, plus nécessairement de bureau. Universités, entreprises, administrations montent des réseaux locaux pour que les utilisateurs puissent avoir un accès rapide à des données centralisées (limitant ainsi l'espace mémoire requis pour leur stockage).

Ces intranets sont souvent reliés entre eux via un réseau mondial, Internet.

2.2 Protocoles de communication

Pour communiquer, les appareils doivent utiliser le même langage, les mêmes règles, scrupuleusement respectées. C'est ce que l'on appelle un protocole. J'utilise la séquence suivante⁴ pour demander à mon fournisseur d'accès une connexion temporaire à Internet :

```
pc @ 19:33:00 ~/usr/tex/html-js # more /etc/ppp/peers/wanadoo.chat
ABORT 'BUSY'
ABORT 'ERROR'
ABORT 'NO CARRIER'
ABORT 'NO DIALTONE'
ABORT 'Invalid Login'
ABORT 'Login Incorrect'
'' 'ATZ'
'OK' 'ATDT0836019301'
CONNECT ''
TIMEOUT 50
ogin: <login_cnx>
ssword: <passwd_cnx>
```

où j'indique quoi faire en cas de réception du signal entre *quotes* dans les premières lignes.

La communication entre ordinateurs a sérieusement commencée (à ma connaissance) avec le protocole *UUCP*⁵ implémenté sur *Unix V7* (Bell Labs. 1977). Cette solution a le défaut d'être lente et consiste essentiellement en la copie de fichiers.

Au début des années 80 l'Université de Berkeley (Californie) a mis au point (dans le cadre d'*ARPANET*) un ensemble de protocoles dit *TCP/IP*⁶ (*Unix BSD 4.2*) qui fait aujourd'hui référence.

³US Defense Advanced Research Projects Agency

⁴Script de connexion par *ppp* sous *NetBSD 1.4.2*

⁵*Unix to Unix Copy Protocol*

⁶*Transfert Control Protocol / Internet Protocol*

Le principe cette fois est d'envoyer non plus des fichiers d'un seul tenant mais de petits paquets que le destinataire reçoit au fur et à mesure et réassemble. On peut ainsi envoyer bien d'autres choses que des fichiers. C'est particulièrement adapté aux réseaux locaux.

La diversité des machines et des systèmes d'exploitation a très vite imposé l'adoption de standards permettant ainsi le partage de données. Aujourd'hui il est commun de lire depuis un *Macintosh* un fichier écrit sur une machine *Quine-daube* et hébergée par une machine *Unix*.

Si *TCP/IP* permet à deux machines de se voir, encore faut-il que l'on puisse lire les données sur les deux. Plusieurs protocoles permettent cela, celui qui nous intéresse plus particulièrement est *HTTP*⁷ (qui fonctionne comme surcouche de *TCP/IP*).

2.3 Le modèle client-serveur

Un utilisateur veut exécuter sur son poste une application. Il se connecte au réseau, appelle le serveur hébergeant l'application, lequel traite la requête en envoyant sur le terminal du client le résultat du programme. Dans ce modèle, il faut voir d'une part, que le serveur fait le gros du boulot et d'autre part, que l'on est tributaire du réseau. Par exemple, pour envoyer un *mail* (via *TCP/IP*) de Paris à Lyon, la communication pourra passer par les États-Unis ou la Suisse selon l'état du trafic et des serveurs relais entre Paris et Lyon.

2.4 Langages à balises et formatage de textes

L'idée remonte là aussi aux années 60. En effet, déjà sous *Multics* (l'ancêtre d'*Unix*) était disponible des mécanismes de formatage de textes via un langage à balises comme *nroff*.

Un informaticien de renom, Donald KNUTH, dégoûté par l'aspect des articles qu'il donnait à publier à l'*American Mathematical Society* se consacra 3 années durant à la mise au point d'un puissant formateur de textes, *T_EX*, particulièrement adapté à la publication d'ouvrages scientifiques. C'est aujourd'hui devenu l'outil préféré des mathématiciens, physiciens et informaticiens qui l'utilisent aussi bien pour leurs photocopies⁸ que pour leurs livres ou articles...

L'idée consiste à décrire à même le texte la forme que devra avoir le résultat final. Ainsi, j'ai tapé le paragraphe précédent sous la forme :

```
L'idée remonte là aussi aux années 60. En effet, déjà sous
\emph{Multics} (l'ancêtre d'\emph{Unix}) était
disponible des mécanismes de formatage de textes via un langage à balises
comme \emph{nroff}.
```

```
Un informaticien de renom, Donald {\sc Knuth}, dégoûté par l'aspect des
articles qu'il donnait publier à l'\emph{American Mathematical Society}
se consacra 3 années durant à la mise au point d'un puissant formateur de
textes, \TeX{}, particulièrement adapté à la publication d'ouvrages
scientifiques. C'est aujourd'hui devenu l'outil préféré des mathématiciens,
physiciens et informaticiens qui l'utilisent pour leurs
photocopies\footnote{celui-ci en est un exemple flagrant {\tt ;-)}} que
pour leurs livres ou articles\ldots
```

Outre *T_EX* et ses dérivés *L^AT_EX*, *L^AT_EX 2_ε*, etc. citons, *HTML*⁹, *SGML*¹⁰ et *XML*¹¹, *me* et *ms* (ensembles de macros dérivés de *roff*) dont voici un exemple :

⁷*Hyper Text Transfert Protocol*

⁸celui-ci en est un exemple flagrant ; -)

⁹*HyperText markup Language*

¹⁰*Standard Generalized Markup Language*

¹¹*eXtended Markup Language*

```
.\" Copyright (c) 1991, 1992, 1996, 1997, 1998 Free Software Foundation
.\" See section COPYING for conditions for redistribution
.TH as 1 "29 March 1996" "cygnus support" "GNU Development Tools"
```

```
.SH NAME
GNU as \- the portable GNU assembler.
```

```
.SH SYNOPSIS
.na
.B as
.RB "[\]" \-a "[\]" dhlns "[\]" \c
\&[\]=\c
.I file\c
\&[\]
.RB "[\]" \-D "[\]"
.RB "[\]" \-\-defsym\ SYM=VAL "[\]"
.RB "[\]" \-f "[\]"
.RB "[\]" \-\-gstabs "[\]"
.RB "[\]" \-I
.I path\c
\&[\]
.RB "[\]" \-K "[\]"
.RB "[\]" \-L "[\]"
.RB "[\]" \-M\ |\ \-\-mri "[\]"
.RB "[\]" \-o
.I objfile\c
\&[\]
.RB "[\]" \-R "[\]"
.RB "[\]" \-\-traditional\-format "[\]"
.RB "[\]" \-v "[\]"
.RB "[\]" \-w "[\]"
.RB "[\]" \-^\- "\ |\" \c
.I files\c
\&[\].\|.|\|.\|]
```

.I i960-only options:

et dont la sortie sur un terminal ASCII donne :

```
pc @ 15:10:01 ~/usr/tex/html-js > man as
```

```
as(1)                                GNU Development Tools                                as(1)
```

```
NAME
GNU as - the portable GNU assembler.
```

```
SYNOPSIS
as [-a[dhlns]=file]] [-D] [--defsym SYM=VAL] [-f] [--gstabs] [-I
path] [-K] [-L] [-M | --mri] [-o objfile] [-R]
[--traditional-format] [-v] [-w] [-- | files...]
```

i960-only options:

Dans tous les cas, il ne s'agit pas de traitement de textes WYSIWYG¹² comme vous les connaissez. On tape le texte et les commandes de formatage dans un fichier texte et ensuite on demande le traitement par un logiciel dédié.

2.5 World Wide Web

En 1989, au CERN, est né le projet du nom *World Wide Web*. Il s'agit à l'origine d'un système de partage d'informations. Il repose essentiellement sur le format HTML. Il s'agit d'afficher dans une fenêtre (ou un écran¹³) du texte, des images, des tableaux... Ajoutons-y encore du son, de la vidéo. La fenêtre est dite une «page». L'HTML est un langage à balises qui permet d'afficher tout cela, de manière structurée et indépendamment de la plate-forme. Il permet aussi de placer des liens au fil du texte, permettant de se connecter à une autre page. Les logiciels chargés d'interpréter l'HTML s'appellent des *browsers*¹⁴.

Ici, le client demande au serveur de lui envoyer une page. Après assemblage des paquets *TCP/IP*, le navigateur interprète le fichier en question et affiche le résultat. Ce modèle a le défaut d'être statique.

2.6 Langages de programmation

Il nous faut enfin parler de langage de programmation pour envisager d'étudier JavaScript et Perl. Les ordinateurs fonctionnent à l'électricité aussi ne comprennent-ils que deux choses : le courant ne passe pas et le courant passe que l'on code par 0 et 1 respectivement. Cela permet la numération en binaire. Des circuits logiques au cœur du processeur se chargent alors des opérations. Comme les humains ont du mal à parler et penser en binaire, ils ont d'abord mis au point des langages dit d'assembleur qui sont très proches du langage machine. Ainsi, pour additionner deux entiers en assembleur on doit écrire quelque chose du genre :

```
MOV @A, R1;
ADD @B, R1;
MOV R1, @C;
```

Puis assez vite sont apparus les premiers langages dits de haut niveau, plus proche du langage humain. Par exemple, pour afficher *Hello World* ! sur un terminal ASCII, on écrira en C :

```
#include <stdio.h>
#include <stdlib.h>

int
main (void)
{
    puts ("Hello World!");
    return EXIT_SUCCESS;
}
```

Dans la majeure partie des cas ces langages sont compilés. Cela a l'avantage de la rapidité d'exécution. En effet, la compilation consiste à traduire les instructions de ces fichiers textes en langage machine et à enregistrer le résultat. Pour exécuter le programme, on demande à la machine de lire le fichier compilé. Votre système d'exploitation est compilé de même que la plupart de vos applications. Les langages compilés les plus courants actuellement sont C, C++, FORTRAN.

Le problème est que pour passer d'une plate-forme à une autre, il faut souvent réécrire le programme avant même de pouvoir le recompiler. C'est là que les langages interprétés trouvent une partie de leur intérêt (pourvu

¹²What You See Is What You Get

¹³Il existe encore des navigateurs en mode texte, le plus connu étant *Lynx*.

¹⁴La traduction la plus commune est «navigateur». D'autres préfèrent dire «butineur».

que les programmes soient portables). Malheureusement ils sont plus lents et nécessitent un interpréteur. Les langages interprétés les plus courants sont Perl, Python, Basic, PHP.

Sun Micro Systems a plus ou moins trouvé la parade avec Java. Il s'agit d'un langage à mis chemin entre la compilation et l'interprétation. Sa portabilité fait son succès depuis quelques années. Évidemment l'avènement d'Internet n'y est pas étranger d'autant plus que les *browsers* sont capables de lancer des *applets*¹⁵.

2.7 Common Gateway Interface versus JavaScript

C'est une interface de programmation, permettant d'obtenir des pages dynamiques. Les langages communément utilisées sont le Perl et le PHP. Dans ce schéma, l'utilisateur interagit avec la page via son *browser*, lequel transmet les requêtes aux serveur. Ce dernier envoie les données reçues à un programme qu'il héberge et renvoie les résultats sous forme de pages HTML (ou de documents PDF ou...). Ici le serveur fait encore tout le boulot, au détriment des requêtes des autres utilisateurs et de la bande passante.

C'est dans ce contexte qu'a été mis au point JavaScript par *Netscape*. JavaScript permet presque la même chose que *CGI* sauf évidemment, tout ce qui nécessite l'accès au serveur (base de données par exemple) ! La différence notoire est que le code JavaScript est exécutée sur le *browser* du client, sans passer par le réseau ni le serveur, déchargeant ceux-ci d'autant.

2.8 Synthèse

Ces quelques rappels disparates étant faits, il nous reste à assembler les morceaux. On a accès à un réseau hétérogène constitué d'un serveur à qui l'on adresse nos requêtes depuis une machine cliente. Ces requêtes sont relayées par d'autres serveurs qui se chargent d'envoyer chacun des paquets à bon port. Dans le cas qui nous occupe, les requêtes consistent en l'envoi et la réception de pages HTML (créées à la volée ou non) et tout ce qui va avec (images, sons, vidéos, PDF, *etc.*).

2.9 <PUB> À propos de ce poly... </PUB>

Ce n'est qu'un *abstract*, une pâle copie, un mauvais plagiat de [Ch²00] dont je me suis plus que largement inspiré... principalement pour le plan. Il est excellent, peut-être un peu technique parfois pour de non-informaticiens mais très bien pensé ; il regorge d'exemples et de captures d'écrans. Je me suis aussi appuyé sur [SE00] pour les questions plus pointues.

WinErr: 002: No Error - Yet.

¹⁵Souvent traduit par «appliquettes», le terme désigne des programmes Java particuliers.

3 HTML élémentaire

3.1 La page minimale

Elle ressemble à ça :

```
<!-- fichier HelloWorld.htm -->
<HTML>
  <HEAD>
    <!-- en-tete : -->
      <TITLE>Hello World !</TITLE>
    </HEAD>
    <BODY>
      <!-- corps du document -->
        Voila ma premiere page.
        <!-- coordonnees de l'auteur ; optionnel -->
          <ADDRESS>
            Pascal CABAUD, email:pascal.cabaud@wanadoo.fr
          </ADDRESS>
        </BODY>
  </HTML>
```

On note :

- des blocs encadrés par des directives,
- ces directives sont entre chevrons (ce sont les *balises*),
- un bloc principal commence par `<HTML>` et finit par `</HTML>`,
- des blocs secondaires dont deux mis en valeur par l'indentation, l'un contenant les en-têtes du document, l'autre le corps,
- des commentaires `<!-- ... -->`,
- un titre,
- un texte,
- un auteur (optionnel).

Les directives sont ici en majuscules mais rien n'y oblige, l'HTML n'est pas *case sensitive* (sensible à la casse). Le texte ne comporte pas d'accents pour une raison qui sera (longuement) évoquée par la suite.

Le titre est important pour plusieurs raisons :

- la fenêtre du navigateur l'affichera,
- il apparaîtra comme nom du signet de cette page,
- ce sera le titre par défaut de l'icône de la fenêtre réduite (*X Window*).

Je n'ai pas encore parlé des signets ; comme pour les adresses postales, on peut garder en mémoire un certain nombre d'adresses Internet dans les navigateurs. La machine enregistre entre autre l'*URL* (qu'elle masque) et le titre (qui sera affiché par défaut). Il est donc important de choisir un titre pertinent.

J'ai dit que l'indentation mettait en valeur certaines parties du texte de notre page. Étant donné que les navigateurs ne tiennent pas compte de la mise en page de votre code, c'est à vous de trouver celle qui vous convient le mieux. On aurait pu écrire :

```
<HTML><HEAD><TITLE>Hello World !</TITLE></HEAD><BODY>Voila ma premiere
page.</BODY></HTML>
```

Néanmoins, vous aurez parfois à être relu par d'autres aussi prenez toujours soin de présenter proprement vos fichiers.

Nous avons maintenant une page minimale.

3.2 Détail sur les balises

Il nous arrivera souvent de vouloir modifier ou spécifier le comportement d'une balise aussi utilise-t-on pour ce faire des attributs. Le format général en est `<BALISE at1=nr at2="truc">` : on sépare les différents attributs par des espaces, les attributs alphanumériques sont entre *double-quotes*.

3.3 Structuration élémentaire

Un texte n'est lisible que si sa structure saute aux yeux.

Titres et sous-titres

```
<!-- fichier Structure.htm -->
<HTML>
  <HEAD>
    <!-- en-tete : -->
    <TITLE>Document</TITLE>
  </HEAD>
  <BODY>
    <!-- corps du document : -->
    <H1>Le titre</H1>
    <H2>Un sous-titre</H2>
    <H3>Une section</H3>
    <H4>Une sous-section</H4>
    <H5>Un paragraphe</H5>
    <H6>Un sous-paragraphe</H6>
    Le texte.
    <!-- une ligne de separation : -->
    <HR>
    <!-- coordonnees de l'auteur : -->
    <ADDRESS>
      Pascal CABAUD, email:pascal.cabaud@wanadoo.fr
    </ADDRESS>
  </BODY>
</HTML>
```

FIG. 1 – Un fichier HTML structuré Structure.htm.

Le titre

Un sous-titre

Une section

Une sous-section

Un paragraphe

Un sous-paragraphe

Le texte.

Pascal CABAUD, email:pascal.cabaud@wanadoo.fr

FIG. 2 – Le résultat de Structure.htm (via Netscape).

Le titre

Un sous-titre

 Une section

 Une sous-section

 Un paragraphe

 Un sous-paragraphe

Le texte.

Pascal CABAUD, email:pascal.cabaud@wanadoo.fr

FIG. 3 – Le résultat de Structure.htm (via Lynx).

En effet, d'une part la structuration de textes est prise en compte par le langage aussi les balises `<Hn>` ($n = 1, \dots, 6$) ont elles été prévues. Les titres ainsi formés sont en gras, la police et la taille sont déterminées en fonction des préférences de l'utilisateur.

Trait horizontal

Dans l'exemple précédent, vous aurez noté le trait marqué par `<HR>`. On peut indiquer les attributs suivants : `<HR SIZE=val>` `val` étant l'épaisseur du trait en pixels,

`<HR WIDTH=val>` `val` étant la longueur du trait exprimée en pourcentage de largeur de page ou en pixels,
`<HR ALIGN=val>` `val` étant l'une des valeurs *LEFT*, *RIGHT* ou *CENTER* (valeur par défaut),
`<HR NOSHADE>` supprimant l'effet de relief.

Sauts de paragraphe

Les navigateurs, ont l'a dit, ignorent l'indentation du fichier source. En effet, ils reviennent à la ligne en fonction de la largeur de la page. Ils convient donc de spécifier chaque retour à la ligne avec la balise vide `<BR>`. Évidemment, `</BR>` n'a pas de sens (même si elle est licite).

Le saut de paragraphe est quant à lui spécifié par `<P>`. Même si cette balise est souvent considérée comme vide, il est parfaitement légitime et licite d'indiquer la fin d'un paragraphe par `</P>`.

Formatage brut

Il arrive parfois que l'on veuille garder le formatage du fichier source. On utilise alors la balise `<PRE>`. Cela permet de faire des tableaux simples ou de l'*ascii art*.

3.4 Un «petit» mot sur les liens

Pour insérer un lien dans le texte, un titre ou que sais-je encore, on utilise :

```
<A HREF="protocole://server.domain/path/to/file">le texte lie</A>
```

La chaîne le texte lié apparaîtra sous-lignée, souvent en bleu (selon les préférences de l'utilisateur) et permettra d'accéder au fichier `/path/to/file` situé sur le serveur `server.domain` via le protocole `protocol` (le plus souvent HTTP (`http://`), FTP (`ftp://`) ou SMTP (`mailto:`), plus rarement NNTP (`news:`, USENET) ou Telnet (`telnet:`)). Cette partie liée s'appelle une ancre. On peut donc améliorer notre page modèle :

```
<!-- fichier Structure1.htm -->
<HTML>
  <HEAD>
    <TITLE>Document</TITLE>
  </HEAD>
  <BODY>
    <HR>
    <ADDRESS>
      Pascal CABAUD,
      <A HREF="mailto:pascal.cabaud@wanadoo.fr">
        pascal.cabaud@wanadoo.fr
      </A>
    </ADDRESS>
  </BODY>
</HTML>
```

On peut aussi définir des liens à l'intérieur d'une même page. Cela permet de faire simplement une table des matières ou des renvois, des notes... Dans ce cas, on utilise le caractère `#` et on utilise l'attribut *NAME*. Voici un petit exemple :

```
<BODY>
...
<P>Pour tout probleme s'adresser a <A HREF="#auteur">moi</A>.
<P>Pour les injures et menaces de mort, s'adresser plutot a
```

```

<B>/dev/null</B> <TT>;-)</TT>.
...
<ADDRESS>
  Copyright Pascal CABAUD, 2000,
  <A HREF="mailto:pascal.cabaud@wanadoo.fr" NAME="auteur">
    <TT>pascal.cabaud@wanadoo.fr<TT>
  </A>
</ADDRESS>
</BODY>

```

3.5 Un mot sur l'encodage

Il existe divers encodages usuels, l'ASCII est le plus fameux (il «ne prend pas en compte» les caractères accentués) mais il en existe une flopée d'autres, tous plus «standards» les uns que les autres. Pour éviter les problèmes, ajoutez la ligne

```
<META HTTP-EQUIV="Content-Type" CONTENT="text/html; charset=iso-8859-1">
```

dans l'en-tête. Elle spécifie le type de données et l'encodage utilisé (ici ISO-8851-1). On reviendra sur les balises <META ...> plus tard.

3.6 Styles élémentaires

Il est possible de mettre du style en italique via <I>...</I>, en gras via ..., souligné via <U>...</U>, style machine à écrire¹⁶ avec <TT>...</TT>.

3.7 Le premier mot de la fin...

On a vu les principaux aspects de l'HTML. Avec ces quelques notions, on peut déjà écrire des pages structurées, intégralement lisibles tant en mode texte que dans un environnement graphique. Le reste n'est (presque) que gadgets encombrants la bande passante.

Pour se mettre en bouche, commencer à mettre en forme `index.htm`, `cours-unix/index.html` et `cours-html/intro.html`. Après une tentative avec *VI*, on utilisera *XEmacs* et *Lynx*.

WinErr: 01F: Reserved for future mistakes of our developers.

¹⁶*Teletype*.

4 Mise en page améliorée

Pour l'instant, nos documents sont plutôt frustrés. On va donc voir en détail comment les rendre plus conviviaux.

4.1 Inclusion d'images

Format d'images Le format GIF s'est imposé comme standard. Néanmoins, on prendra garde au fait que c'est un format propriétaire. Autrement dit, sans s'attarder sur le problème des brevets de logiciels, disons que *British Telecoms* qui possède le brevet est en droit de vous demander des *royalties* si vous l'utilisez. Étant donné la position de l'Europe face au problème il vaut mieux éviter soigneusement ce format (à moins d'être sûr que l'on utilise un logiciel pour lequel la situation a été clarifié avec *British Telecoms*, comme c'est le cas pour le serveur HTTP *Apache*).

L'autre format (non-propriétaire celui-là) et que l'on préférera est le JPEG. Le PNG est supporté avec plus ou moins de bonheur par les navigateurs.

Les trois permettent une bonne compression de l'information mais néanmoins une petite image fait tout de suite plusieurs dizaines voire centaines de Ko.

Utilisateurs et bande passante La connexion disponibles dans les universités est en général de type Ethernet 10-BaseT ou 100-BaseT en intranet, le tout relié à une fibre optique (débit quasiment infini). Il n'y a aucune commune mesure entre le débit obtenu et par exemple celui d'une connexion par modem 56K, ces derniers étant environ 10 fois plus lents.

Je vous rappelle donc que tout le monde ne dispose pas d'une connexion haut-débit, gratuite ou illimitée et que la majorité des utilisateurs paient leur connections à internet comme des appels téléphoniques au tarif local. Tout cela pour dire que, quand bien même les images donnent un attrait particulier à vos pages HTML, elles les allourdissent considérablement, donc allongent le temps de chargement et encombrant la bande passante (gênant ainsi tous les utilisateurs du réseau).

Enfin, rappelez-vous que certains lisent l'HTML en mode texte et n'ont que faire des poissons rouges et maisons sur la colline et que d'autres ont de vieux modems 28K ou moins.

On utilise la balise `<IMG . . . >` comme suit :

```
<IMG SRC=/home/pc/pub/images/truc.jpg>
```

mais rien n'oblige à ce que l'image soit sur le même serveur que le source HTML auquel cas :

```
<IMG SRC="http://www.bertillon.fr/~pc/images/machin.jpg">
```

Il est possible de spécifier divers attributs :

- **ALIGN=va l**, pour indiquer la mise en page de l'image dans du texte ; les valeurs possibles pour **va l** sont : **TOP** (le haut de l'image coïncide avec le haut du texte), **MIDDLE** (le milieu de l'image...) et **BOTTOM** (le bas de l'image coïncide avec le bas du texte).
- **ALT**, pour donner un commentaire lisible dans un navigateur en mode texte.
- **WIDTH** et **HEIGHT** permettent de spécifier la taille apparente de l'image (en points).
- **BORDER** pour mettre un cadre (épaisseur en points à spécifier).
- les attributs **LEFT**, **RIGHT**, **VSPACE** et **HSPACE** permettent de couler une image dans un paragraphe. Pour chacun, il s'agit de la distance en points entre l'image et le texte.

```
<!-- fichier Image.htm -->
<HTML>
  <HEAD>
    <!-- en-tete : -->
```

```

<TITLE>Exemple d'image sur une page</TITLE>
<META HTTP-EQUIV="Content-Type" CONTENT="text/html; charset=iso-8859-1">
</HEAD>
<BODY>
<!-- corps du document -->
  <BLOCKQUOTE>
    <BLOCKQUOTE>
      <P><B>BSD Daemon Copyright 1988 by Marshall Kirk McKusick.<BR>
      </B>All Rights Reserved</P>
    <BLOCKQUOTE>
      <P>Permission to use the daemon may be obtained from:</P>
      <BLOCKQUOTE>
        <P>Marshall Kirk McKusick<BR>
        1614 Oxford St.<BR>
        Berkley, CA 94709-1608</P>
      </BLOCKQUOTE>
      <P>or via email at
      <A HREF="mailto:mckusick@mckusick.com">mckusick@mckusick.com</A>
    </BLOCKQUOTE>
  </BLOCKQUOTE>
  <IMG SRC="daemon.jpg" ALIGN=LEFT HSPACE=5 VSPACE=5
  BORDER=2 ALT="Image de Chuck">
  <P>Je n'ai pas grand'chose a dire pour meubler quelques lignes
  donc je me force un peu pour ne rien dire. McKusick n'est pas tres
  content que son oeuvre ait ete reprise par des commerciaux aussi
  impose-t-il cette notice chaque fois que <I>Chuck</I> (le nom de
  la mascotte BSD ci-contre, aussi appelee <I>Beastie</I>) apparait.
  C'est un petit diabolotin car en
  1988 deja les <B>daemons</B> avaient ete implements sur BSD.
  Il s'agit de programmes qui fonctionnent en tache de fond attendant
  les requetes (programmes serveurs souvent). La bonne traduction
  serait <B>elfe</B> mais tout le monde a compris <B>demon</B>. Pour
  une fenetre de taille normale, il me faut encore quelques lignes que
  je tente desesperement de combler ainsi...
  <P>
  <!-- coordonnees de l'auteur -->
  <ADDRESS>
    Copyright Pascal CABAUD .<->*//<->*//<->*//<->*//<->*//<->*//<-
  >*//© , 2001,
    <A HREF="mailto:pascal.cabaud@wanadoo.fr">
      <TT>pascal.cabaud@wanadoo.fr</TT>
    </A>
  </ADDRESS>
</BODY>
</HTML>

```

On pourra aussi (surtout avec mon exemple) mettre une image en guise d'ancree. Par exemple ici, on mettrait plutôt l'image comme lien sur le fichier de license de MCKUSICK. Ça donne :

```
<A HREF="copyright.htm"><IMG SRC="daemon.jpg" ALT="BSD Daemon"></A>
```

BSD Daemon Copyright 1988 by Marshall Kirk McKusick.
All Rights Reserved

Permission to use the daemon may be obtained from:

Marshall Kirk McKusick
1614 Oxford St.
Berkley, CA 94709-1608

or via email at mckusick@mckusick.com



Je n'ai pas grand'chose à dire pour meubler quelques lignes donc je me force un peu pour ne rien dire. McKusick n'est pas très content que son oeuvre ait été reprise par des commerciaux aussi impose-t-il cette notice chaque fois que *Chuck* (le nom de la mascotte BSD ci-contre, aussi appelée *Beastie*) apparaît. C'est un petit diabolotin car en 1988 déjà les **daemons** avaient été implémentés sur BSD. Il s'agit de programmes qui fonctionnent en tâche de fond attendant les requêtes (programmes serveurs souvent). La bonne traduction serait **elfe** mais tout le monde a compris **démon**. Pour une fenêtre de taille normale, il me faut encore quelques lignes que je tente désespérément de combler ainsi...

Copyright Pascal CABAUD©, 2000, pascal.cabaud@wanadoo.fr

FIG. 4 – Le résultat de `Image.htm` (vu avec *Netscape*).

où le fichier `copyright.htm` contient les lignes idoines.

4.2 Listes

Listes descriptives

Elles permettent de faire un glossaire par exemple. Elle se présente sous la forme d'*item* mis en valeur par des puces, l'indentation et/ou les attributs de la fonte utilisée (graisse, chasse, etc.).

Une liste descriptive est déclarée par `<DL>` et `</DL>`. On peut faire figurer la première ligne descriptive des *items* par l'attribut `COMPACT` (fonctionne sur certains *browsers* seulement). Chaque *item* est déclaré par la balise vide `<DT>`, les descriptions sont marquées par `<DD>`.

Listes régulières

Il y a deux types de listes : les listes numérotées et les autres ; pour une liste numérotée on utilise `<OL>`/`</OL>`, pour une liste à puces on utilise `<UL>`/`</UL>`. Pour donner un titre à une liste on utilise la balise vide `<LH>`. Chaque élément est marqué par la balise vide `<LI>`. Elle admet l'attribut `TYPE` qui prend les valeurs `square`, `circle` et `disc` pour les listes non ordonnées, `1`, `I`, `i`, `a` pour les autres. On peut évidemment imbriquer des listes.

```
<DL>
  <DT>Un <I>item</I> :
    <DD>Sa description.
  <DT>Un autre :
    <DD>La description de l'autre.
</DL>

<DL>
  <DT COMPACT>Autre exemple avec l'attribut <TT>COMPACT</TT>.
  <DD>Description correspondante.
</DL>

<HR>
<OL>
  <LH> <H2>Titre d'une liste imbriquée</H2>
  <LI TYPE=I>Premier élément.
  <UL>
    <LH> <H3>Sous-titre</H3>
    <LI TYPE=square>Premier élément de la "sous-liste".
    <LI TYPE=disc>Second.
    <LI TYPE=circle>Troisième.
  </UL>
  <LI TYPE=i>Second élément.
  <LI TYPE=A>Troisième élément.
  <LI TYPE=1>Quatrième élément.
  <LI TYPE=a>Cinquième élément.
</OL>
```

4.3 Tableaux

Pour déclarer un tableau on utilise `<TABLE>`/`</TABLE>`. Cette balise admet les attributs suivants (en *pixels* d'écran) :

- `BORDER` fixe l'épaisseur du bord extérieur ;

- *CELLPADDING* fixe l'écart entre le bord de la cellule et le texte qui est dedans (en bas à gauche de la cellule pour une cellule alignée à gauche);
- *CELLSPACING* fixe l'épaisseur du filet entre les cellules;
- *WIDTH* fixe la largeur du tableau par rapport à la largeur de la page. Ici la valeur est un pourcentage (`<TABLE WIDTH="100%">`).

Lignes et cellules

La gestion des tableaux dans un langage à balises est un peu difficile pour le néophyte. En effet, il faut organiser le tableau ligne par ligne.

Une ligne d'un tableau se déclare par la balise `<TR>`/`</TR>`. Une cellule d'une ligne se déclare par `<TD>`/`</TD>`.

On peut spécifier les attributs de ces balises :

- *ALIGN* pour aligner le texte à gauche (`<TR ALIGN=left>`), à droite (`<TR ALIGN=right>`) ou centré (`<TR ALIGN=center>`);
- *VALIGN* pour aligner verticalement le texte en haut (`<TR VALIGN=top>`), en bas (`<TR VALIGN=bottom>`) ou centré (`<TR VALIGN=middle>`).

Certaines cellules peuvent être des en-têtes, on utilise alors `<TH>`/`</TH>` (avec les mêmes attributs que `<TD>`/`</TD>`).

Couleur d'un tableau, d'une ligne ou d'une cellule Les balises `<TABLE>`, `<TR>` et `<TD>` admettent toutes trois l'attribut *BGCOLOR* qui permet de changer la couleur de fond.

Subtilités et horreurs

On peut vouloir avoir une cellule faisant deux fois (ou autant que vous voulez) la hauteur (ou largeur) des autres (ou seulement certaines d'entre elles). Cela se fait *via* les attributs *COLSPAN* et *ROWSPAN* des balises `<TR>` et `<TD>`.

Pour qu'une cellule apparaisse sur une seule ligne, ajoutez l'attribut *NOWRAP* à `<TD>`.

```
<TABLE WIDTH="80%" BORDER=5 CELLPADDING=10 CELLSPACING=2>
  <CAPTION ALIGN=bottom>
    Legende
  </CAPTION>
  <TR BGCOLOR="#ff0000">
    <TH ALIGN=left>Premiere colonne
    <TH>Seconde colonne
    <TH>Troisieme colonne
  </TR>
  <TR>
    <TD ALIGN=center>Premiere cellule normale
    <TD BGCOLOR="#00ff00">Seconde cellule normale
    <TD>Troisieme cellule normale
  </TR>
</TABLE>
```

```
<HR>
<TABLE BORDER=6 CELLPADDING=10 CELLSPACING=6>
  <TR ALIGN=center>
    <TD>
    <TD>
    <TD>1ere ligne, 3eme colonne
```

```

        <TD>1ere ligne, 4eme colonne
    </TR>
    <TR ALIGN=center>
        <TD ROWSPAN=2 COLSPAN=2>Sur 2 lignes, deux 1eres colonnes
        <TD>2eme ligne, 3eme colonne
        <TD>2eme ligne, 4eme colonne
    </TR>
</TABLE>

```

4.4 Polices, couleurs, tailles, styles

Polices, couleurs et taille

Pour modifier ces attributs, on dispose de la balise `<FONT>`/`</FONT>`. Elle admet les attributs :

- `COLOR` pour indiquer la couleur dans le système RGB ¹⁷ ;
- `FACE` attribut non-standard ;
- `SIZE` sur une échelle de 1 à 7. Précédé d'un +, on incrémente la taille de la police de la valeur indiquée à partir de la taille courante, précédé d'un -, on décrémente.

Ainsi, on pourra taper :

```
<FONT COLOR="#0ff000" SIZE=+2>en gros</FONT>
```

Styles

Il y a huit styles :

- `<CITE>` Pour faire une citation, souvent en italique.
- `<CODE>` Pour montrer du code, souvent en chasse fixe (télétype).
- `<DFN>` Pour introduire une définition, souvent en gras, éventuellement italique.
- `<EM>` Pour mettre en valeur du texte (*emphasis*).
- `<KBD>` Pour montrer ce qu'il faut entrer au clavier, souvent en chasse fixe.
- `<SAMP>` Pour un exemple, souvent en chasse fixe.
- `<STRONG>` Pour mettre en valeur, souvent en gras.
- `<VAR>` Pour une variable, souvent en italique.

En fait la notion de style en HTML est plus compliquée que ce que je vous ai présenté jusqu'à maintenant. Pour faire court, disons qu'il y a 3 styles :

1. le style physique, il s'agit des balises `<B>`/`</B>`, `<I>`/`</I>`, `<TT>`/`</TT>` et `<U>`/`</U>` ;
2. le style logique dont on vient de parler ;
3. le style de paragraphe avec `<ADDRESS>`/`</ADDRESS>` et `<PRE>`/`</PRE>` dont on a déjà parlé et `<BLOCKQUOTE>`/`</BLOCKQUOTE>`.

4.5 Derniers détails de mise en page

Le multi-colonnage

On fait ça avec `<MULTICOL>`/`</MULTICOL>`. Les attributs possibles sont :

- `COLS`, le nombre de colonnes ;
- `GUTTER`, le nombre de *pixels* de largeur de gouttière¹⁸ ;
- `WIDTH`, le pourcentage de largeur de la page occupée par les colonnes (et les gouttières).

¹⁷ Red-Green-Blue, au format `rrggbb`, avec `r`, `g`, `b` ∈ {0, 1, ..., 9, A, B, ..., F}, ie. numéroté en hexadécimal.

¹⁸ L'espace entre les colonnes

Fonds d'écran et autres gadgets

La balise `<BODY>` admet les attributs *BGCOLOR* (pour la couleur de fond, en *RGB* toujours) et *BACKGROUND* (pour le fond, donné par une image).

On peut vouloir aussi modifier (en jouant sur les attributs de `<BODY>` toujours) d'un bout à l'autre de la page les couleurs du texte normal (attribut *TEXT*, en *RGB*), des liens non-encore suivis (attribut *LINK*), que l'on est en train de suivre (attribut *ALINK*) et qui ont déjà été suivis (attributs *VLINK*).

WinErr: 020: Error recording error codes. Additional errors will be lost.

5 Frames

Le principe est simple : on divise la fenêtre du navigateur en plusieurs zones contenant chacune une page distincte. L'intérêt est évident : on peut ainsi avoir des *frames* contenant table des matières, références, images, etc. que l'on ne charge qu'une fois et d'autres contenant le reste. Ça permet d'alléger la bande passante et d'éviter d'avoir à revenir en arrière à longueur de temps. Malheureusement, il reste des navigateurs incapables de gérer (correctement) les *frames*.

Les exemples suivants ont été testés tant bien que mal et le navigateur *Opera* 5.0b6 s'est beaucoup mieux acquitté de la tâche que le bon vieux *Netscape Communicator* 4.73. Les navigateurs en mode texte *Lynx* et *W3m* ne supportent pas les *frames*.

5.1 Déclaration

Il faut connaître trois balises :

- `<FRAMESET>`/`</FRAMESET>` qui sert à délimiter les *frames* ;
- `<FRAME>`/`</FRAME>` qui sert à indiquer au navigateur le contenu de la *frame* courante ;
- `<NOFRAMES>`/`</NOFRAMES>` qui sert à indiquer aux navigateurs ne gérant pas les *frames* ce qu'ils ont à afficher.

Une page déclarant des *frames* ne contient pas de balises `<BODY>`/`</BODY>`.

```
<HTML>
  <HEAD>
    <TITLE>Document avec Frames</TITLE>
  </HEAD>
  <FRAMESET ROWS="*,*" FRAMEBORDER=yes>
    <FRAME SRC="Frame1.htm" NAME=Titre MARGINWIDTH=5 MARGINHEIGHT=5>
  <FRAMESET ROWS="*,*" FRAMEBORDER=yes>
    <FRAME SRC="Frame2.htm" NAME=Corps MARGINWIDTH=5 MARGINHEIGHT=5>
  </FRAMESET>
  <NOFRAMES>
    Cette page contient des <EM>frames</EM>, que votre navigateur
    ne peut afficher.
  </NOFRAMES>
</HTML>
```

5.2 Les attributs de `<FRAMESET>`

`<FRAMESET>` admet :

- `BORDER` (si `FRAMEBORDER=no`) qui vaut un nombre de *pixels* séparant les *frames* ;
- `BORDERCOLOR` (si `FRAMEBORDER=yes`) en *RGB* ;
- `COLS` et `ROWS` qui s'affectent comme suit :

```
<FRAMESET ROWS="50%, 25%, 10%, 15%" COLS="100%">
...
<FRAMESET ROWS="*, 2*">
```

- `FRAMEBORDER` qui vaut *no* ou *yes*.

Avec les maigres explications, précédentes, vous devriez vous douter qu'entre `<FRAMESET>` et `</FRAMESET>` il ne peut y avoir que les balises `<FRAME>`/`</FRAME>` et `<NOFRAMES>`/`</NOFRAMES>`.

FRAME admet :

- *MARGINHEIGHT* nombre de *pixels* entre la haut de la *frame* et le haut du texte de cette même *frame* ;
- *MARGINWIDTH* *idem* pour la largeur (à gauche si le texte est aligné à gauche) ;
- *NAME* nom de la zone. Vous ne pouvez pas donner de nom commençant par un *_* (*underscore*) ;
- *NORESIZE* empêche toute modification de la taille de la *frame* ;
- *SCROLLING* qui vaut *yes* ou *no* selon que l'on veut une barre de défilement ou non ;
- *URL* fichier à charger dans la *frame*.

Il y a encore quelques autres options moins usitées.

WinErr: 01B: Illegal Error. You are not allowed to get this error. Next time you will get a penalty for that.

6 *Cascading Style Sheets*

On a dit et on est maintenant convaincu que l'HTML se concentre surtout sur le fond, sur le contenu du texte plutôt que sur la forme. Si l'on veut une mise en page de type PAO, il faut passer à la vitesse supérieure. Une norme a donc été proposée en 1997 par le W3C¹⁹ mais elle n'est pas encore totalement supportée par les navigateurs actuels.

Les exemples suivants ont été testés tant bien que mal et le bon vieux *Netscape Communicator* 4.73 s'est beaucoup mieux acquitté de la tâche que le navigateur *Opera* 5.0b6. Les navigateurs en mode texte *Lynx* et *W3m* ne supportent pas les CSS.

6.1 Usage, types

On peut définir un style à même un document HTML en le définissant dans l'entête :

```
<HEAD>
...
<STYLE TYPE="text/css">
<!--
    H1 {color: "#0000ff"}
    H2 {color: "#00ff00"}
-->
</STYLE>
...
</HEAD>
```

ou en créant des pages spécifiques. Par exemple, pour redéfinir les titres, je pourrais avoir :

```
<pc @ zeus 0:32:58 ~/public_html/styl/> cat styl.css
H1, H2, H3 {text-align=center; color="#ff0000"}
H4, H5, H6 {color="#0000ff"}
```

On les utilise de deux manières, **importées** ou **liées**. Dans le premier cas, il s'agit d'ajouter une balise `<LINK>` dûment remplie dans l'entête :

```
<LINK REL=stylesheet
      TYPE="text/css"
      HREF="http://www.bertillon.fr/styl.css"
      TITLE="mon style">
```

tandis que dans le second, il s'agit de compléter un style par une autre feuille en utilisant la commande `@IMPORT URL ( )` ; comme suit :

```
<STYLE>
<!--
...
@IMPORT URL(http://www.bertillon.fr/styl.css);
@IMPORT URL(../styl/mon_styl.css);
...
-->
</STYLE>
```

¹⁹World Wide Web Consortium, cf. <http://www.w3.org>.

La subtile différence réside dans le fait que dans le second cas des règles d'un premier style peuvent être écrasées, modifiées ou complétées par les règles suivantes. En effet, on peut utiliser plusieurs balises `<LINK>` dans un même fichier. Dans ce cas, soit il est proposé un choix à l'utilisateur (et le navigateur n'utilise alors que la feuille choisie) soit ces dernières sont fusionnées, la dernière ayant autorité. En principe, la dernière solution est supportée par les dernières versions des navigateurs en vue.

6.2 Syntaxe, classes de styles et héritage

Une règle a au moins trois éléments :

- une liste de sélecteurs de balises,
- des propriétés,
- et leur valeurs.

Propriété et valeur sont séparés par des `:`, les couples propriétés/valeurs par des `;`, les sélecteurs sont séparés par des `,`.

On peut spécifier des sélecteurs contextuels. Le plus souvent cela permet de définir des propriétés de balises lorsqu'elles sont emboîtées. Par exemple²⁰ :

```
OL LI          {list-style: upper-alpha} /* lettres majuscules */
OL OL LI       {list-style: upper-roman} /* chiffres romains majuscules */
OL OL OL LI    {list-style: lower-alpha} /* lettres minuscules */
OL OL OL OL LI {list-style: lower-roman} /* chiffres romains minuscules */
```

Quand on définit les propriétés d'une balise, si des propriétés ont déjà été spécifiées, soit cette définition contredit la précédente auquel cas la dernière fait autorité, soit cette nouvelle définition ne fait qu'affiner les précédentes spécifications.

À la manière de la programmation par objet, une notion de classe de style a été mise au point permettant ainsi l'héritage. Une classe de style permet de définir des styles différents pour une même balise. Il y a plusieurs types de classes de style :

- **les classes régulières** qui servent par exemple à définir différents styles de paragraphes ou de titre ; par exemple :

```
<STYLE>
...
H1.centre {text-align: center}
H1.droite {text-align: right}
...
</STYLE>
...
<H1 CLASS=centre>Titre centre</H1>
...
<H1 CLASS=droite>Titre aligne a droite</H1>
...
```

- **les classes génériques** qui sont indépendantes de la balise ; par exemple :

```
<STYLE>
...
.centre {text-align: center}
</STYLE>
...
```

²⁰Pompé sur [SE00].

```

<H1 CLASS=centre>Titre centre</H1>
...
<P CLASS=centre>textes du paragraphe centre.</P>
...

```

- **les identificateurs comme classes** : il s'agit de jouer sur *ID*. Cet attribut générique définit un identificateur (unique dans le document) pour la balise concernée. On utilise alors # pour y faire référence :

```

<STYLE>
...
#centre {text-align: center}
H2#droite {text-align: right}
...
</STYLE>
...
<H1 ID=centre>Titre centre</H1>
...
<H2 ID=droite>Titre aligne a droite</H2>
...

```

C'est un *hack infect*, il vaut mieux éviter ce truc.

- **les pseudo-classes**, *mon-gros-seau-d'eau* équivalentes aux classes, elles diffèrent en deux points : on sépare l'identificateur de pseudo-classe de la balise par : et les identificateurs sont prédéfinis *A:link*, *A:active*, *A:visited*, **:first-line* et **:first-letter*. Il faut très bien connaître le langage pour s'en servir donc je ne m'étends pas, vous renvoyant à [SE00].

Comme les possibilités des CSS sont larges, je vous renvoie aux normes, disponibles sur <http://www.w3.org> pour les détails.

WinErr: 003: Dynamic linking error. Your mistake is now in every file.

7 Compléments divers

Après discussion avec vos enseignants, il sera peu question de JavaScript dans ce poly.

Comme dit Emmanuel DREYFUS,

« JavaScript est encapsulé dans HTML, qui encapsulait déjà pas mal d'autres conneries comme ça. »

Bref, ce n'est pas la panacée. Néanmoins, c'est incontournable pour faire des choses un peu sérieuses comme ouvrir une nouvelle fenêtre, gérer des événements dans des fenêtres différentes ou des choses plus basiques comme une calculatrice francs-euros... Un des intérêts de JavaScript est qu'il peut permettre de tester les arguments que l'on va envoyer à un *CGI*, allégeant ainsi bande passante et serveur.

7.1 Ouvrir et fermer une nouvelle fenêtre...

JavaScript est langage interprété, qui s'inspire des langages à objets, dont le code est inséré à même la page HTML, dans l'entête. Par la suite on appelle les fonction dans le code HTML, *via* des attributs de balise. On peut mettre le code dans un fichier `file.js` et en spécifier *URL* et contenu dans l'entête, en attribut de `<SCRIPT>/<SCRIPT>` comme dans l'exemple suivant :

```
<HTML>
  <HEAD>
    <TITLE>Lien ouvrant une fenetre</TITLE>
    <SCRIPT
      SRC="file:/home/pc/usr/tex/html-js/Script.js"
      LANGUAGE=JavaScript>
    </SCRIPT>
  </HEAD>
  <BODY>
    Ceci <A HREF="Page.htm" TARGET="Fenetre"
      onclick="ouvrirFenetre()">est un lien</A>
      vers une autre page.
  </BODY>
</HTML>
```

Le fichier `Script.js` contient :

```
function ouvreFenetre () {
  refwin = window.open ("", "Fenetre")
}

function fermerFenetre () {
  window.close()
}
```

Quand on clique sur le lien, une seconde fenêtre (classe *window*) s'ouvre (méthode *open()*) affichant la page `Page.htm`. En effet, la fenêtre s'appelle *Fenetre*, à savoir la valeur de *TARGET*. Il est possible de :

1. critiquer ce code ; en effet, pourquoi tenir compte du résultat de l'appel à *window.open()* si on ne s'en sert plus après ? Et pourquoi a-t-on déclaré *ouvrirFenetre()* si celle-ci ne fait qu'appeler *window.open()* ?
2. spécifier des variables supplémentaires à *window.open()*, permettant de masquer les différentes barres d'outils du navigateur, définir la taille de la fenêtre...

Voici les deux pages correspondantes :

```

<HTML>
  <HEAD>
    <TITLE>Lien ouvrant une fenetre</TITLE>
    <SCRIPT
      SRC="file:/home/pc/usr/tex/html-js/Script.js"
      LANGUAGE=JavaScript>
    </SCRIPT>
  </HEAD>
  <BODY>
    Ceci <A HREF="Page.htm" TARGET="Fenetre"
      onclick="ouvrirFenetre()">est un lien</A>
    vers une autre page.
  </BODY>
</HTML>

<HTML>
<HEAD>
  <TITLE>La fameuse autre page</TITLE>
  <SCRIPT
    SRC="file:/home/pc/usr/tex/html-js/Script.js"
    LANGUAGE=JavaScript>
  </SCRIPT>
</HEAD>
<BODY>
  <P>Il n'y a pas grand'chose sur cette page...</P>
  <FORM METHOD="get">
    <INPUT TYPE="button"
      onclick="fermerFenetre()"
      VALUE="Fermer">
  </FORM>
</BODY>
</HTML>

```

7.2 Envoyer un mail

Il y a la méthode `<A HREF="mailto:mr.sylvestre@worldcompany.com">...</A>` que l'on a déjà vu. Après le chapitre sur les CGI, on peut raffiner comme suit :

```

<A HREF="mailto:mr.sylvestre@worldcompany.com?Subject=Stock%20options">
  J'en veux plus !</A>

```

Si on veut quelque chose de réellement convivial, à même une page HTML, mieux vaut utiliser un formulaire :

```

<FORM METHOD="post"
  ACTION="mailto:mr.sylvestre@worldcompany.com"
  ENCTYPE="text/plain">
  <TEXTAREA NAME="courrier" ROWS="3" COLS="50"></TEXTAREA>
  <INPUT TYPE="submit" VALUE="Envoyer">
  <INPUT TYPE="reset" VALUE="Annuler">
</FORM>

```

WinErr: 004: Erroneous error. Nothing is wrong.

8 Bibliographie

Hormis [Ch²00] (*cf.* 2.9, très bien pour apprendre), il n’y a pas grand’chose de réellement bien chez les libraires. Pour aller plus loin, je vous recommande vivement de garder [SE00] sous le coude, il est exhaustif (par contre, pour apprendre c’est inutilisable). Les autres éditeurs font des choses très bien pour caler un meuble bancal ou pour faire bien sur une étagère mais guère plus en général.

Quiconque voudra apprendre la programmation telle que les barbus la pratique se plongera dans [K&R90], c’est l’ouvrage de référence. Certains lui préfèrent [Bra98] pour sa rigueur et son exhaustivité. J’attire votre attention sur le point suivant : les professionnels disent que quiconque sait bien programmer en C sait bien programmer tout court. Il vaut bien mieux souffrir un peu en faisant (sérieusement) du C qu’avoir l’impression de «faire des choses» avec *Visual Machin* (les mauvaises habitudes sont toujours faciles à prendre et difficiles à perdre). Au pire, on pourra se mettre à Perl (en lisant [SC98] ; la référence absolue [Camel Book] est abordable avec des notions), lequel est plus adapté que le C pour des CGI pas trop gourmands en temps de calcul. Un module pour *Apache* est prévu à cet effet.

Pour ceux qui veulent approfondir la notion de réseau TCP/IP, [Hun98] est une bonne ressource. Pour les futurs *webmestres*, la lecture de [Lau²99] s’impose.

Références

- [Bra98] BRAQUELAIRE J.-P. (1998), *Méthodologie de la programmation en langage C, Bibliothèque standard – API POSIX*, 3^{ème} édition, 2^{ème} tirage, coll. Enseignement de l’Informatique, Masson, Paris.
- [Ch²00] CHALÉAT P. & CHARNAY D. (2000), *Programmation HTML et JavaScript*, 2^{ème} édition, 6^{ème} tirage, Eyrolles, Paris.
- [Hun98] HUNT C. (1998), *TCP/IP, Administration de réseaux*, 2^{ème} édition française, O’Reilly, Paris.
- [K&R90] KERNIGHAN B. W. & RITCHIE D. M. (1990), *Le langage C, norme ANSI*, 2^{ème} édition, coll. Informatiques, Masson, Paris.
- [Kir99] KIRCH O. (1999), *Administration réseau sous Linux*, édition française revue et corrigée, O’Reilly, Paris.
- [Lau²99] LAURIE B. & LAURIE P. (1999), *Apache, installation et mise en œuvre*, 2^{ème} édition française, O’Reilly, Paris.
- [SC98] SCHWARTZ R. L. & CHRISTIANSEN T. (1996), *Introduction à Perl*, 2^{ème} édition française, O’Reilly, Paris.
- [SE00] SPAINHOUR S. & ECKSTEIN R. (2000), *Webmaster in a nutshell, Manuel de référence*, 2^{ème} édition française, O’Reilly, Paris.
- [Camel Book] WALL L., CHRISTIANSEN T. & SCHWARTZ R. L. (1996), *Programmation en Perl*, 2^{ème} édition française, O’Reilly, Paris.

De plus, ces machines sont sous des OS multitaches (ou tache tout court pour celles qui tournent sous NT)

-- JLP in Guide du Macounet Pervers : Bien NToyer ses tâches --